

1 A feladat megoldása

1.1 A probléma áttekintése

A feladat a következőképpen fogalmazható meg: n darab különböző csoport áll rendelkezésünkre, amelyekbe r darab azonos tárgyat kell elhelyezni. Az első csoportnak nem üresnek kell lennie (azaz $st = r$), és pontosan k darab nem üres csoportot kell létrehozni, ahol $st \leq k \leq dr$. Határozzuk meg az összes lehetséges elhelyezések számát.

1.2 Kulcsfontosságú megfigyelések

Az r azonos tárgy n különböző csoportba való elhelyezését két lépésre bonthatjuk:

1. Válasszuk ki azokat a k csoportot, amelyek nem üresek maradnak: $\binom{n}{k}$ lehetőség.
2. Osszuk el a maradék $r - k$ tárgyat a k kiválasztott csoport között tet-szőlegesen (beleértve a 0-t is), mivel mindegyik már tartalmaz legalább egyet.

A második lépés az stars and bars probléma: $\binom{(r-k)+k-1}{k-1} = \binom{r-1}{k-1}$ lehetőség.

Tehát az első pillanatban a válasz $\sum_{k=st}^{dr} \binom{n}{k} \binom{r-1}{k-1}$ lenne. Azonban ez nem veszi figyelembe, hogy a csoportok belső struktúrája nem lényeges - a csoportok csak azt számítja, hogy melyikük üresek és melyikük nem.

1.3 Stirling számok alkalmazása

A helyes megoldás a Stirling számok másodfajú $S(m, k)$ használatát igényli, amely az m elem k darab nem üres részhalmazra való particionálásainak számát adja meg.

Átalakítás után a következő formula adódik:

$$\text{Ans} = \sum_{k=st}^{dr} \binom{n}{k} \cdot S(n + r - 1, k)$$

Itt $m = n + r - 1$, ami a tárgyak és csoportok összessége (az első csoport már nem üres, így $r - 1$ tárgy marad).

A Stirling számok másodfajú definíciója:

$$S(m, k) = \frac{1}{k!} \sum_{i=0}^k (-1)^{k-i} \binom{k}{i} i^m$$

1.4 A precomputáció

A megoldás $O(n)$ precomputációt használ a faktoriálisok és moduláris inverzek előzetes kiszámítására:

```
void pre(int en) {
    fact[0] = fact[1] = 1LL;
    ifact[0] = ifact[1] = 1LL;
    inv[0] = inv[1] = 1LL;
    for (ll i = 2; i <= en; i++) {
        fact[i] = (fact[i-1] * i) % mod;
        inv[i] = (inv[mod % i] * (mod - mod / i)) % mod;
        ifact[i] = (ifact[i-1] * inv[i]) % mod;
    }
}
```

Ez lehetővé teszi a binomiális együtthatók gyors kiszámítását: $\binom{n}{k} = \frac{n!}{k!(n-k)!}$.

1.5 A pref táblázat

A kód egy speciális prefix tömböt épít:

$$pref[k] = \sum_{j=0}^k \frac{(-1)^j}{j!} \pmod{mod}$$

Ez váltakozva adja hozzá és vonja ki az inverz faktoriálisokat:

$$\begin{aligned} pref[0] &= 1 \\ pref[1] &= 1 - \frac{1}{1!} = 0 \\ pref[2] &= 0 + \frac{1}{2!} = \frac{1}{2} \\ pref[3] &= \frac{1}{2} - \frac{1}{3!} = \frac{1}{3} \end{aligned}$$

Ez a tömb lehetővé teszi a hatékony tartományösszeg queries-t a végső ciklusban.

1.6 Paraméterek transzformálása

A kód a bemeneti paramétereket transzformálja az algoritmus egyszerűsítése érdekében:

```
n -= r;
n++;
st -= r;
st++;
dr -= r;
dr++;
```

Ez biztosítja, hogy a Stirling számok argumentumai és a tartományok megfelelően legyenek kezelve.

1.7 Összegzés és végső képlet

A fő ciklus a következő összeget számítja:

$$\text{Ans} = \sum_{i=0}^{dr} \frac{i^n}{i!} \cdot (\text{pref}[dr - i] - \text{pref}[\max(0, st - i - 1)]) \pmod{\text{mod}}$$

Ez egyesíti a Stirling számok formuláját a prefix tömbbel, lehetővé téve az $O(dr)$ időben történő kiszámítást.

1.8 Idő- és memória komplexitás

- **Idő komplexitás:** $O(n + dr)$, ahol $n \leq 100000$.
- **Memória komplexitás:** $O(n)$, három tömböt használva: fact, ifact, inv.

1.9 Megoldási stratégia összefoglalása

1. Precomputáljuk a faktoriálisokat és moduláris inverzeket $O(n)$ időben.
2. Építsük fel a pref tömböt váltakozó előjellel.
3. Számítsuk ki a Stirling számokat a végső ciklusban az inclusion-exclusion elv segítségével.
4. Adjuk meg az eredményt modulo $10^9 + 7$.